Algoritmo e Laboratório de Programação II

Aula 11 – Operações com Arquivos – Parte 2 (Nível Avançado)

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Gravando estruturas em disco

- Se o arquivo conterá estruturas, gostaríamos de gravar ou ler uma por vez.
- Para gravar ou ler estruturas, utilizaremos as mesmas funções do exemplo anterior, fread() e fwrite().
- Criaremos um programa para gravar registros de livros de uma biblioteca.

Gravando estruturas em disco

Prog11.c

```c
/* Prog11.c */
/* Grava estruturas em disco */
#include <stdio.h> /* define FILE */
#include <stdlib.h>
#include <conio.h> /* Para getch() */

typedef struct Livro
{
    char Titulo[50];
    char Autor[50];
    short NumReg;
    double Preco;
} Livro;

Livro GetLivro()
{
    Livro livro;
    printf("\n\tDigite o título: ");
    gets(livro.Titulo);
    printf("\tDigite o nome do autor: ");
    gets(livro.Autor);
    printf("\tDigite o número do registro: ");
    scanf("%hd",&livro.NumReg);
    printf("\tDigite o preco: ");
    scanf("%lf", &livro.Preco);
    rewind(stdin);
    return livro;
}
```

Gravando estruturas em disco

```
30  int main()
31  {
32      Livro livro;
33      char resp;
34      FILE *fptr; /* ponteiro para arquivo */
35
36      if((fptr = fopen("Livros.Dat","wb"))==NULL) exit(1);
37
38      do
39      {
40          livro = GetLivro();
41          if(fwrite(&livro,sizeof(Livro),1,fptr)!=1) break;
42          printf("Mais um livro (s/n)? ");
43          resp = getche();
44      }
45      while(resp != 'n' && resp != 'N');
46
47      fclose(fptr);
48      system("pause");
49      return 0;
50  }
```

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A Função rwind()

- Usamos essa função em GetLivro() para limpar alguma sobra de bytes no teclado antes da próxima leitura.
- Seu protótipo é o seguinte:

```
void rewind(FILE *stream);
```

- Essa função reposiciona o indicador de posição de arquivo no início do arquivo especificado pelo argumento.

A Função rwind()

- A instrução:

```
rewind(stdin);
```

- Usa o ponteiro FILE stdin associado ao teclado.
- Lendo Estruturas do Disco:
  - O programa a seguir lê objetos gravados pelo programa anterior:

A Função rwind()

Prog13.c

```c
/* Prog13.c */
/* Lê estruturas do disco */
#include <stdio.h> /* define FILE */
#include <stdlib.h>
#include <conio.h> /* Para getch() */

typedef struct Livro
{
    char Titulo[50];
    char Autor[50];
    short NumReg;
    double Preco;
} Livro;

void PrintLivro(Livro livro)
{
    printf("\n\tTítulo: %s\n", livro.Titulo);
    printf("\tAutor : %s\n", livro.Autor);
    printf("\tNo.Reg: %hd\n", livro.NumReg);
    printf("\tPreço : %.2lf\n", livro.Preco );
}
```

A Função rwind()

```
23  int main()
24  {
25      Livro livro;
26      char resp;
27      FILE *fptr; /* ponteiro para arquivo */
28
29      if((fptr = fopen("Livros.Dat","rb"))==NULL) exit(1);
30
31      while(fread(&livro, sizeof(Livro),1,fptr)==1) PrintLivro(livro);
32
33      fclose(fptr);
34      system("pause");
35      return 0;
36  }
```

A Função rwind()

- Lendo Estruturas do Disco:
  - Para que os programas de leitura e gravação de objetos trabalhem corretamente é necessário que a seção de dados seja a mesma tanto na leitura quanto na gravação.
  - As variáveis do tipo Livro dos nossos exemplos têm exatamente 110 bytes de tamanho, sendo que os primeiros 50 bytes são ocupados por uma cadeia de caracteres que representam o título de um livro, os proximos 50 bytes representam o nome do autor, os outros dois, o número do registro do livro e os últimos oito, o preço do livro.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Gravando e Lendo de Um Mesmo Arquivo

- O nosso próximo programa trabalha no mesmo arquivo tanto para gravação como para leitura.
- O programa adiciona quantos livros o usuário desejar e depois lê e imprime o conteúdo total do arquivo.
- Eis a listagem:

Gravando e Lendo de Um Mesmo Arquivo

Prog14.c

```c
/* Prog14.c */
/* Grava e lê estruturas em disco */
#include <stdio.h> /* define FILE */
#include <stdlib.h>
#include <conio.h> /* Para getch() */

typedef struct Livro
{
    char Titulo[50];
    char Autor[50];
    short NumReg;
    double Preco;
} Livro;

Livro GetLivro()
{
    Livro livro;
    printf("\n\tDigite o título: ");
    gets(livro.Titulo);
    printf("\tDigite o nome do autor: ");
    gets(livro.Autor);
    printf("\tDigite o número do registro: ");
    scanf("%hd",&livro.NumReg);
    printf("\tDigite o preço: ");
    scanf("%lf", &livro.Preco);
    rewind(stdin);
    return livro;
}
```

Gravando e Lendo de Um Mesmo Arquivo

```
36  }
37
38  int main()
39  {
40      Livro livro;
41      char resp;
42      FILE *fptr; /* ponteiro para arquivo */
43
44      /* Abre arquivo para leitura e gravação. Adiciona dados no final*/
45      if((fptr = fopen("Livros.Dat","ab+"))==NULL) exit(1);
46
47      do
48      {
49          livro = GetLivro();
50          if(fwrite(&livro,sizeof(Livro),1,fptr)!=1) break;
51          printf("Mais um livro (s/n)? ");
52          resp = getche();
53      }
54      while(resp != 'n' && resp != 'N');
55
56      fflush(fptr); /* Esvazia o conteúdo do buffer de saída */
57      fseek(fptr,0,0); /*Coloca o ponteiro no início do arquivo */
58
59      puts("\n\nLISTA DE LIVROS DO ARQUIVO");
60      puts("==========================");
61
62      while(fread(&livro, sizeof(Livro),1,fptr)==1) PrintLivro(livro);
63      fclose(fptr);
64      system("pause");
65      return 0;
66  }
```

Gravando e Lendo de Um Mesmo Arquivo

- A primeira novidade nesse programa é o modo usado para abrir o arquivo.
- A instrução:

- Abre o arquivo para leitura e gravação.
- Usamos a para preservar as estruturas que já existem no arquivo e acrescentar novas ao final dele.

Gravando e Lendo de Um Mesmo Arquivo

- Se o arquivo não existir, será criado.
- Usamos + pois queremos executar as duas operações: de gravação e de leitura.
- Usamos b para trabalhar em modo binário.
- O programa grava uma estrutura por vez usando a função fwrite().

Gravando e Lendo de Um Mesmo Arquivo

- Após o termino da gravação das estruturas digitadas, desejamos ler o arquivo inteiro e imprimir seus dados.
- Antes da leitura, esvaziamos o buffer de saída por meio da instrução:

```
fseek(fptr,0,
```

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A Função fflush()

- A função fflush() grava o conteúdo de qualquer dado existente no buffer para o arquivo associado ao ponteiro que recebe como argumento.
- Retorna 0 para indicar sucesso, caso contrário, retorna EOF.
- Após esvaziar o buffer, devemos modificar a posição corrente do arquivo para o seu inicio.
- Fazemos isso na instrução:

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A Função fseek()

- Para que você possa compreender como esse programa trabalha, é necessário familiarizar-se com o conceito de ponteiros para arquivos.
- Um ponteiro para um arquivo aponta para um byte particular chamado posição atual.
- Todas as funções que mostramos neste capítulo precisam conhecer o ponteiro para o arquivo que desejamos manipular.

A Função fseek()

- A cada tempo em que gravamos ou lemos qualquer coisa no arquivo, o ponteiro é movido para o fim dessa coisa e a próxima leitura ou gravação começará nesse ponto.
- Quando o arquivo é aberto, o ponteiro é fixado em seu primeiro byte; então, se quisermos ler ou gravar nele estaremos no seu início.

A Função fseek()

- Se abrirmos o arquivo usando a opção a (append), o seu ponteiro será posicionado no fim do arquivo.
- A função fseek() permite movimentar a posição corrente de leitura e gravação do arquivo para uma posição escolhida.
- A instrução:
- Fixa o ponteiro no primeiro byte do arquivo.

A Função fseek()

- Como a leitura do nosso arquivo deve começar no seu início, queremos deslocar zero bytes a partir do início do arquivo.
- A função fseek() aceita três argumentos.
- O primeiro é um ponteiro para a estrutura FILE do arquivo. Após a chamada a fseek(), esse ponteiro será movimentado para a posição que desejarmos.

A Função fseek()

- O segundo argumento é chamado deslocamento, e consiste no número de bytes que desejamos deslocar a partir da posição especificada pelo terceiro argumento.

A Função fseek()

- O terceiro argumento chama-se posição. Existem três possibilidades para esse número, as quais determinam de onde o deslocamento começará a ser medido, a saber:

Posição	Significado
0	Início do arquivo
1	Posição atual
2	Fim do arquivo

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A Função ftell()

- A função ftell() retorna a posição corrente do ponteiro de um arquivo (em bytes), sempre a partir do início do arquivo.
- Essa função aceita um único argumento, que é o ponteiro para a estrutura FILE do arquivo, e retorna um valor do tipo long, que representa o número de bytes do começo do arquivo até a posição atual.

A Função ftell()

- A função ftell() pode não retornar o número exato de bytes se for usada com arquivos em modo texto.
- Lembre-se de que, em modo texto, a combinação CR/LF é representada por um único caractere em C.

A Função ftell()

- Calculando o número de registros do arquivo:
  - Para mostrar o uso das funções fseek() e ftell(), escreveremos um programa que calcula o número de registros do arquivo.
  - O programa pergunta para o usuário qual é o número de registro que ele quer ver, procura os dados do registro escolhido no arquivo e imprime o seu conteúdo.
  - Veja a listagem:

A Função ftell()

Prog15.c

```c
/* Prog15.c */
/* Mostra o uso de fseek() e ftell() */
#include <stdio.h> /* define FILE */
#include <stdlib.h>
#include <conio.h> /* Para getch() */

typedef struct Livro
{
    char Titulo[50];
    char Autor[50];
    short NumReg;
    double Preco;
} Livro;


void PrintLivro(Livro livro)
{
    printf("\n\tTítulo: %s\n", livro.Titulo);
    printf("\tAutor : %s\n", livro.Autor);
    printf("\tNo.Reg: %hd\n", livro.NumReg);
    printf("\tPreco : %.2lf\n", livro.Preco );
}
```

A Função ftell()

```
24  int main()
25  {
26      Livro livro;
27      char resp;
28      FILE *fptr; /* ponteiro para arquivo */
29      long nrec, rec, posicao;
30
31      if((fptr = fopen("Livros.Dat","rb"))==NULL) exit(1);
32
33      fseek(fptr,0,2);
34
35      nrec = ftell(fptr)/sizeof(Livro);
36      printf("O número de registros é %ld\n", nrec);
37
38      puts("Insira o número do registro: ");
39      scanf("%ld", &rec);
40      if(rec > 0 && rec <= nrec)
41      {
42          posicao = (nrec-1) * sizeof(Livro);
43          fseek(fptr,posicao,0);/* Posiciona no registro desejado */
44          fread(&livro, sizeof(Livro),1,fptr);
45          PrintLivro(livro);
46          fclose(fptr);
47      }
48      else
49          puts("Registro não existe");
50
51      system("pause");
52      return 0;
53  }
```

A Função ftell()

- Calculando o número de registros do arquivo:
  - Observe que o número de registros começa em 1 para o usuário e em 0 para o programa.
  - Assim, na instrução:

```
posicao = (nrec-1) * sizeof(Livro);
```

  - Corrigimos a diferença.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Imprimindo na Impressora

- Imprimir dados na impressora é exatamente a mesma coisa que gravar dados num arquivo em disco.
- O sistema operacional define o nome de alguns periféricos ligados ao computador.
- Qualquer um deles pode ser usado em nossos programas como nomes de arquivos.

Imprimindo na Impressora

- A tabela seguinte descreve estes nomes:

Nome	Descrição
COM	Console (teclado e vídeo).
AUX ou COM1	Primeira porta serial.
COM2	Segunda porta serial.
PRN ou LPT1	Primeira impressora paralela.
LPT2	Segunda impressora paralela.
LPT3	Terceira impressora paralela.
NULL	Periférico inexistente.

- O programa seguinte usa o nome PRN para abrir a primeira impressora paralela e imprime nela o conteúdo do arquivo especificado na linha de comando.

Imprimindo na Impressora

Prog16.c

```c
/* Prog16.c */
/* Imprime esse arquivo na impressora */
#include <stdio.h> /* define FILE */
#include <stdlib.h>

int main()
{

    char buff[80];
    FILE *fptr, *fprn; /* ponteiro para arquivo */

    if((fptr = fopen("FilePrint.C","r"))==NULL)
    {
        puts("Arquivo FilePrint.C não foi encontrado");
        exit(1);
    }

    if((fprn = fopen("PRN","w"))==NULL) /* Abre arquivo impressora */
    {
        puts("Impressora indisponivel.");
        exit(1);
    }

    while(!feof(fptr))
    {
        fgets(buff,80,fptr); /*Lê uma linha de texto */
        fputs(buff,fprn);    /*Imprime na impressora */
    }
    fclose(fptr);
    fclose(fprn);
    system("pause");
    return 0;
}
```

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Condições de Erro

- Na maioria dos casos, se um arquivo for aberto com sucesso, poderemos ler ou escrever nele.
- Existem situações, entretanto, em que isso não acontece.
- Um problema de hardware pode ocorrer enquanto a leitura ou gravação estiver sendo executada;

Condições de Erro

- O espaço final em disco, enquanto gravamos, pode ser insuficiente ou pode ocorrer qualquer outro problema.
- Em nossos programas, assumimos que semelhantes erros de leitura e gravação não ocorrem.
- Mas em situações reais, como programas em que a integridade dos dados é crítica, pode ser desejável a verificação explícita de erros.

Condições de Erro

- Várias funções de E/S não retornam um erro explícito.
- Por exemplo, se fputc() retornar EOF, isso pode indicar tanto um EOF como um erro.
- Para determinar se um erro ocorreu, podemos usar a função ferror().
- Tal função retorna verdadeiro se tiver ocorrido um erro durante a última operação no arquivo ou, caso contrário, retorna zero (falso).

Condições de Erro

- O seu único argumento é um ponteiro file.
- Outra função útil em conjunto com ferror() é perror().
- Seu único argumento é uma mensagem de erro que indica em que parte do programa ocorreu.
- A mensagem é impressa seguida de vírgula e da mensagem de erro do sistema.

Condições de Erro

- Se um arquivo foi aberto para gravação, podemos implementar o seguinte trecho de código:

```
if (ferror(fptr)){
    /* Verifica erro */
    perror("Erro de Gravação: ");
    fclose(fptr);
    exit(1);
}
```

- Se, por exemplo, for detectado um erro de disco, ferror() retornará um valor verdadeiro (não zero) e perror() imprimirá a seguinte mensagem

Erro de gravação: Bad data

Condições de Erro

- A primeira parte da mensagem é fornecida pelo programa, e a segunda, pelo sistema operacional.
- Mensagens de erros explicitas podem ser informativas para o usuário e para o programador durante o desenvolvimento do programa.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Leitura e Gravação em Baixo-Nível

- A leitura e gravação em baixo-nível oferece somente duas funções: read() e write().
- Em baixo-nível, os dados não podem ser lidos ou gravados como caracteres individuais, strings ou formatados como em alto-nível.
- Existem somente um modo de ler e gravar dados: por meio de um buffer cheio de dados.

Leitura e Gravação em Baixo-Nível

- Gravar ou ler um buffer completo de dados se assemelha à maneira de ler e gravar blocos de bytes em alto-nível ou também chamado leitura e gravação bufferizada.
- Entretanto, a leitura e gravação não bufferizada, ou baixo-nível, obriga o programador a criar e manter o buffer de dados.
- As funções não farão essa manutenção.

Leitura e Gravação em Baixo-Nível

- Há algumas vantagens em usar o modo não bufferizado, pois possibilita a criação de funções menores e mais rápidas.
- As funções de alto-nível foram criadas usando as funções de baixo-nível.
- Baixo-nível é o método básico de leitura e gravação, então, as funções de baixo-nível são usadas para a criação de funções de alto-nível.

Leitura e Gravação em Baixo-Nível

- As funções de baixo-nível podem acessar os arquivos de entrada padrão, saída padrão e a saída de erros usando os seguintes descritores de arquivos:
  - Stdin 0
  - Stdout 1
  - Stderr 2
- Imprimindo no vídeo e lendo do teclado em baixo-nível:

Leitura e Gravação em Baixo-Nível

Prog17.c

```c
/* Prog17.c */
/* Lê de stdin e imprime em stdout usando baixo nível */
#include <io.h> /* para read() e write() */
#include <stdlib.h> /* para system() */
#include <string.h> /* para strlen() */

int main()
{
    char *str="Digite o nome seu nome: ";
    char nome[80];
    int BytesLidos;

    write(1,str,strlen(str)); /* Imprime em stdout */

    BytesLidos=read(0,nome,80); /* Lê de stdin */

    nome[BytesLidos + 1]='\0';

    str="Seu nome é ";
    write(1,str,strlen(str)); /* Imprime em stdout */
    write(1,nome,strlen(nome));
    system("pause");
    return 0;
}
```

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A função write()

- A função write() recebe três argumentos: o descritor do arquivo no qual iremos gravar, o endereço do início do buffer que queremos gravar e o número de bytes a serem gravados. Nosso programa grava no arquivo stdout (video) por meio da instrução:

```
write(1,str,strlen(str)); /* Imprime em stdout */
```

- A função write() retorna o número de bytes realmente gravados.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A função read()

- A função read() recebe três argumentos: o descritor do arquivo no queremos ler, o endereço do início do buffer em que iremos colocar os dados lidos e o número máximo de bytes a serem lidos. Nosso programa lê arquivo stdin (teclado) por meio da instrução:

```
BytesLidos=read(0,nome,80); /* Lê de stdin */
```

- A função read() retorna o número de bytes realmente lidos.

A função read()

- Esse é um número importante, visto que pode muito bem ser menor que o número de bytes especificado no terceiro argumento.
- Precisamos saber justamente quanto foi lido antes de podermos fazer qualquer coisa com seu conteúdo.
- Atribuímos esse número à variável BytesLidos.
- O caractere \0 não é adicionado, fazemos isso na instrução: `nome[BytesLidos + 1]='\0';`

A função read()

- Lendo em baixo-nível de arquivos em disco:
  - O nosso primeiro exemplo é um programa que abre e lê um arquivo em disco e, em seguida, imprime-o no vídeo.
  - Usaremos o arquivo informado na linha de comandos:

A função read()

Prog18.c

```c
/* Prog18.c */
/* Lê e imprime arquivo usando baixo nível */
#include <io.h> /* para read() e write() */
#include <stdlib.h> /* para exit() */
#include <string.h> /* para strlen() */
#include <fcntl.h> /* necessario para flags */
#define TAMBUFF 1024 /* tamanho do buffer */
char buff[TAMBUFF + 1];

int main(int argc, char **argv)
{
    char *str="Erro: Digite o nome do arquivo na linha de comandos\n";
    int fd, BytesLidos;

    if(argc !=2)
    {
        write(1,str,strlen(str)); /* Imprime em stdout */
        exit(1) ;
    }

    if( (fd = open(argv[1], O_RDONLY | O_BINARY)) < 0)
    {
        str = "Arquivo não existe!";
        write(1,str,strlen(str)); /* Imprime em stdout */
        exit(1);
    }
```

A função read()

```
28      while((BytesLidos = read(fd,buff,TAMBUFF)) > 0)
29      {
30          buff[BytesLidos + 1] = '\0';
31          write(1,buff,strlen(buff));
32      }
33      close(fd);
34      return 0;
35  }
```

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Fixando o Tamanho do Buffer

- Em baixo-nível, nossos programas devem manter a área de memória em que os dados lidos do arquivo serão colocados.
- Essa área de memória é chamada de buffer.
- As instruções para criar o buffer de caracteres são:

```
#define TAMBUFF 1024 /* tamanho do buffer */
char buff[TAMBUFF + 1];
```

Fixando o Tamanho do Buffer

- O tamanho do buffer é importante para uma operação eficiente.
- Dependendo do sistema operacional, buffers de certos tamanhos são manuseados de modo mais eficiente que outros.
- Geralmente, o tamanho ótimo de buffer é múltiplo de 512 bytes.
- Portanto 512, 1024, 2048, ou 4096 podem ser usados.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A Função open()

- Antes de poder acessar um arquivo em baixo-nível, você deve abri-lo usando a função open().
- Seu primeiro argumento é o nome completo (com caminho) do arquivo que queremos abrir.
- Já o segundo argumento indica o modo em que o arquivo será acessado.

A Função open()

- Cada modo é indicado por uma constante chamada flag, cujo valor está definido no arquivo fcntl.h.
- Para informar as nossas opções entre as possibilidades de abertura do arquivo (leitura, gravação, modo texto, modo binário) devemos combinar múltiplos flags por meio do operador OR (|) bit-a-bit.

A Função open()

- A tabela seguinte descreve essas combinações de flags:

Flag	Significado
O_APPEND	Posiciona o ponteiro de arquivo no fim dele.
O_CREAT	Cria o arquivo, caso não exista.
O_RDONLY	Abre o arquivo somente para leitura.
O_WRONLY	Abre o arquivo somente para gravação.
O_RDWR	Abre o arquivo somente para leitura e gravação.
O_TRUNC	Abre e trunca o conteúdo de um arquivo existente para o tamanho zero.
O_BINARY	Abre arquivo no modo binário.
O_TEXT	Abre arquivo no modo texto.

A Função open()

- Algumas das possibilidades listadas são mutuamente exclusivas, por exemplo, você não pode abrir o arquivo para somente-leitura e somente-gravação ao mesmo tempo.
- O terceiro argumento é requerido somente quando O_CREAT é especificado.

A Função open()

- Se o arquivo já existir, esse argumento é ignorado.
- Caso contrário, especifica os flags de permissão que serão selecionados quando o novo arquivo for gravado pela primeira vez.
- Quando abrimos um arquivo com open(), as permissões selecionadas na criação do arquivo são aplicadas.
- Essas permissões estão definidas no arquivo sys\stat.h.

A Função open()

- S_IREAD: Permissão somente para leitura.
- S_IWRITE: Permissão para gravação.
- S_IREAD|S_IWRITE: Permissão para leitura e gravação.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

O descritor de arquivo

- Observe que o arquivo stdio.h, que era necessário em alto-nível, não o é aqui.
- Ao contrário de arquivos em alto-nível que usam um ponteiro FILE para a comunicação entre o sistema operacional e o seu arquivo, em baixo-nível requer-se um descritor de arquivo.

O descritor de arquivo

- Se tudo ocorrer com sucesso, a função open() retornará um número inteiro chamado descritor de arquivo.
- Caso ocorra um erro, retornará -1 e definirá a variável global errno com um dos seguintes valores:
  - EACCES: Permissão de acesso negada.
  - EEXIST: Arquivo já existe.
  - EINVAL: código de acesso (flags) inválido.
  - EMFILE: Arquivos demais abertos.
  - ENOENT: Arquivo não existe.

O descritor de arquivo

- Lendo com read()
  - Nosso arquivo foi aberto para leitura em binário.
  - Depois de aberto, o programa lê seus dados por meio da instrução:

```
28      while((BytesLidos = read(fd,buff,TAMBUFF)) > 0)
29      {
30          buff[BytesLidos + 1] = '\0';
31          write(1,buff,strlen(buff));
32      }
```

  - O programa coloca as informações contidas no arquivo na matriz buff[].
  - Após cada leitura, adicionamos o caractere zero na matriz e imprimimos o seu conteúdo no vídeo.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A função close()

- Usamos close() para fechar o arquivo, liberando, assim, as áreas de comunicação para serem usadas por outros arquivos.
- A função close() é chamada com um único argumento: o descritor do arquivo.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Mensagens de erros

- Podemos usar a função perror() para verificar o tipo de erro ocorrido:

```
if( (fd = open(argv[1], O_RDONLY | O_BINARY)) < 0)
{
    perror("Não foi possível abrir o arquivo para leitura!");
    exit(1);
}
```

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A função lseek()

- Nossos exemplos acessam arquivos para leitura e gravação de modo sequencial, usando as técnicas de baixo-nível.
- Programas mais sofisticados precisam ser capazes de ler e gravar arquivos de modo aleatório.
- A função lseek() é semelhante à função fseek(), que permite movimentar a posição corrente de leitura e gravação do arquivo para uma posição escolhida.

A função lseek()

- A função lseek() aceita três argumentos.
- O primeiro argumento é o descritor do arquivo.
- O segundo é o número de bytes que desejamos deslocar a partir da posição especificada pelo terceiro argumento; e este, por sua vez, é chamado posição.

A função lseek()

- Existem três possibilidades para esse número que determinam de onde o deslocamento começará a ser medido.

Posição	Significado
0	Início do arquivo
1	Posição atual
2	Fim do arquivo

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

A função tell()

- A função tell() retorna a posição corrente de um arquivo (em bytes), sempre a partir do inicio do arquivo.
- Essa função aceita um único argumento – o descritor do arquivo - e retorna um valor do tipo long, que representa o número de bytes do começo do arquivo até a posição atual.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Operando com buffers

- A técnica de guardar o conteúdo de um arquivo em um buffer proporciona várias vantagens.
- Por exemplo, podemos executar várias operações nos dados do buffer sem ter de acessar o arquivo novamente.
- A biblioteca C oferece algumas funções com a finalidade de otimizar operações em dados bufferizados.

Operando com buffers

- O programa procura uma palavra, fornecida pelo usuário na linha de comandos, em um arquivo-texto em disco.

Prog19.c

```
1   /* Prog19.c */
2   /* Procura frase no arquivo usando baixo nível */
3   #include <io.h> /* para read() e write() */
4   #include <stdlib.h> /* para exit() */
5   #include <string.h> /* para strlen(), memchr() */
6   #include <fcntl.h> /* necessario para flags */
7   #include <stdio.h> /* para NULL etc. ..*/
8
9   #define TAMBUFF 1024 /* tamanho do buffer */
10  char buff[TAMBUFF];
```

Operando com buffers

Prog19.c

```
12  /* procura() */
13  /* procura frase no buffer */
14  void procura(char *frase, int tambuf)
15  {
16      char *ptr,*p;
17      ptr = buff;
18      while((ptr = memchr(ptr,frase[0],tambuf)) != NULL)
19          if(memcmp(ptr,frase,strlen(frase)) == 0)
20          {
21              puts("Primeira ocorrencia da frase: ");
22              for(p = ptr- 100; p < ptr+100; p++)
23                  putchar(*p);
24              exit(1);
25          }
26          else
27              ptr++;
28  }
```

Operando com buffers

```
30  int main(int argc, char **argv)
31  {
32      char *str="Formato: Procura NomeArq.xxx frase\n";
33      int fd, BytesLidos;
34
35      if(argc != 3)
36      {
37          write(1,str,strlen(str)); /* Imprime em stdout */
38          exit(1) ;
39      }
40
41      if( (fd = open(argv[1], O_RDONLY )) < 0)
42      {
43          str = "Arquivo não existe!";
44          write(1,str,strlen(str)); /* Imprime em stdout */
45          exit(1);
46      }
47
48      while((BytesLidos = read(fd,buff,TAMBUFF)) > 0)
49          procura(argv[2],BytesLidos);
50      close(fd);
51      str = "Frase nao encontrada\n";
52      write(1,str,strlen(str)); /* Imprime em stdout */
53      return 0;
54  }
```

Operando com buffers

- O programa requer que o usuário digite três argumentos na linha de comando: o nome do programa, o nome do arquivo em que quer procurar e a frase a ser procurada.
- A frase, na verdade, é uma única palavra, mas o programa pode ser modificado para aceitar uma frase, desde que se concatenem os argumentos da linha de comando que a representam.

Operando com buffers

- O programa procura a primeira ocorrência da frase.
- Para mostrar onde a frase se encontra no arquivo, são impressos 100 caracteres de cada lado da frase.
- O nosso arquivo é aberto em modo texto, que é o modo padrão e não necessita ser especificado.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Funções para manipulação de buffers: memchr() e memcmp()

- A função procura() chama duas funções de manipulação de buffer.
- A primeira delas é memchr().
- Essa função procura no buffer por um caractere específico.

```
(ptr = memchr(ptr,frase[0],tambuf))
```

- Essa instrução mostra os três argumentos necessários para a chamada a memchr().

Funções para manipulação de buffers: memchr() e memcmp()

- O primeiro argumento é o endereço do buffer onde queremos procurar.
- O segundo, é o caractere a ser procurado, neste caso o primeiro caractere da frase que o usuário digitou na linha de comandos (argv[2]).
- O terceiro parâmetro é o tamanho do buffer a ser procurado.

Funções para manipulação de buffers: memchr() e memcmp()

- A função memchr() retorna NULL, se não tiver encontrado o caractere; caso contrário, retorna um ponteiro para o caractere no buffer.
- Aqui, o ponteiro é atribuído à variável ptr.
- A função procura() executa então um comando if para verificar se o caractere marca, de fato, o começo da frase.

Funções para manipulação de buffers: memchr() e memcmp()

- Esta comparação é executada pela função memcmp(), na expressão:

```
if(memcmp(ptr,frase,strlen(frase)) == 0)
```

- Esta função também recebe três argumentos: o ponteiro para o lugar do buffer onde a comparação deve começar, o endereço da frase a ser comparada e o comprimento da frase.
- A função compara, então, a frase com os caracteres do buffer e, se for igual, retorna 0.

Funções para manipulação de buffers: memchr() e memcmp()

- Se a frase for encontrada, procura() imprime caracteres dos dois lados da frase e termina.
- Caso contrário, o processo é repetido.
- As funções de manipulação de buffer requerem que o arquivo string.h seja inserido no programa.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Gravando Arquivos em baixo-nível

- Gravar arquivos em baixo-nível é um pouco mais complicado que ler arquivos.
- Como exemplo, escreveremos um programa que copia um arquivo em outro, isto é, que imita o comando COPY da console.
- O usuário digita na linha de comandos o nome do arquivo de origem, que deve existir em disco, e o nome do arquivo destino, que será criado.

C>fcopy origem.txt dest.txt

Gravando Arquivos em baixo-nível

- Visto que arquivos são abertos em modo binário, qualquer arquivo pode ser copiado: tanto arquivos-texto como programas executáveis, ou arquivos em binário.

Prog20.c

```
1   /* Prog20.c */
2   /* Copia um arquivo em outro, usando baixo nível */
3   #include <io.h> /* para read() e write() */
4   #include <stdlib.h> /* para exit() */
5   #include <string.h> /* para strlen(), memchr() */
6   #include <fcntl.h> /* necessario para flags */
7   #include <sys\stat.h> /* para permissao */
8
9   #define TAMBUFF 1024 /* tamanho do buffer */
10  char buff[TAMBUFF];
```

Gravando Arquivos em baixo-nível

```
12  int main(int argc, char **argv)
13  {
14      int fdOrigem, fdDest, BytesLidos;
15      char *str="Formato: Fcopy ArqOrigem.xxx ArqDest.xxx\n";
16
17      if(argc != 3)
18      {
19          write(1,str,strlen(str)); /* Imprime em stdout */
20          exit(1) ;
21      }
22      if( (fdOrigem = open(argv[1], O_RDONLY | O_BINARY)) < 0)
23      {
24          str = "Arquivo não existe!";
25          write(1,str,strlen(str)); /* Imprime em stdout */
26          exit(1);
27      }
28      if((fdDest = open(argv[2], O_CREAT | O_TRUNC | O_WRONLY |
29                        O_BINARY , S_IWRITE))< 0)
30      {
31          str = "Nao posso abrir o arquivo de gravação.";
32          write(1,str,strlen(str)); /* Imprime em stdout */
33          exit(1);
34      }
35      while((BytesLidos = read(fdOrigem,buff,TAMBUFF)) > 0)
36          write(fdDest,buff,BytesLidos);
37      close(fdOrigem);
38      close(fdDest);
39      return 0;
40  }
```

Gravando Arquivos em baixo-nível

- Dois arquivos são abertos.
- Um é aquele do qual leremos, e o outro é arquivo que será criado e gravado.
- Seus descritores são, respectivamente, as variáveis fdOrigem e fdDest.
- A expressão que abre o arquivo origem é familiar, mas a que abre o arquivo destino tem aspecto diferente.

Gravando Arquivos em baixo-nível

```
if((fdDest = open(argv[2], O_CREAT | O_TRUNC | O_WRONLY |
                  O_BINARY , S_IWRITE))< 0)
```

- Para criar um novo arquivo, usamos a flag O_CREAT.
- Desse modo, a função open() requer um terceiro argumento, a permissão.
- Queremos a permissão para gravar, assim usamos S_IWRITE.

Roteiro

- Gravando estruturas em disco
- A função rewind()
- Gravando e lendo de um mesmo arquivo
- A função fflush()
- A função fseek()
- A função ftell()
- Imprimindo na impressora
- Condições de erro
- Leitura e gravação em baixo-nível
- A função write()
- A função read()
- Fixando o tamanho do bufer
- A função open()
- O descritor de arquivo
- A função close()
- Mensagens de erros
- A função lseek()
- A função tell()
- Operando buffers
- Funções para manipulação de buffers: memchr() e memcmp()
- Gravando arquivos em baixo-nível
- Quando usar o quê?

Quando usar o quê?

- Pela multiplicidade de funções oferecidas para acessar arquivos em C, é, às vezes, difícil saber que método usar.
- Leitura e gravação em alto-nível é, provavelmente, o mais valioso quando manuseamos dados como caracteres, strings ou formatados, como em printf().
- Leitura e gravação em baixo-nível é um método mais natural em situações em que blocos são trabalhados, como matrizes, por exemplo.

Quando usar o quê?

- Em muitos casos, alto-nível é mais facil de programar, enquanto baixo-nível gera um código mais eficiente em termos de rapidez e tamanho de código executável.
- É importante não misturar funções de alto-nível com funções de baixo-nível.

Quando usar o quê?

- Se um arquivo foi aberto com fopen(), não use read() ou write() com ele e vice-versal, porque, enquanto baixo-nível é usado para ler ou gravar blocos de dados, fread() e fwrite() são usadas em arquivos que são trabalhados por outras funções de alto-nível.
- Modo texto é usado quando o arquivo contém texto; e modo binário, quando o arquivo pode conter números e outros códigos ASCII. Isso evita problemas de transformação errada de dados numéricos.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;